

02_mysql_relations_joins_beginner

Семинар: Сбор данных из различных источников. Основы работы с SQL

Файл 2. Связи между таблицами, подзапросы, UNION и JOIN

Этот файл продолжает первый материал. В первой части мы создали базу данных `my_database` и таблицу `users`. Теперь добавим вторую таблицу `orders`, свяжем её с пользователями и научимся получать данные сразу из нескольких таблиц.

Главная идея этого файла: в реальных базах данных информация редко хранится в одной большой таблице. Обычно данные разделены по смыслу: пользователи отдельно, заказы отдельно, товары отдельно, платежи отдельно. SQL позволяет связывать такие таблицы между собой.

0. Что мы изучим в этом файле

После прохождения материала слушатель сможет:

- понять, зачем нужны связи между таблицами;
- создать таблицу `orders` для заказов пользователей;
- понять роль `PRIMARY KEY` и `FOREIGN KEY`;
- добавить данные в связанную таблицу;
- использовать подзапросы;
- объединять результаты через `UNION`;
- соединять таблицы через `INNER JOIN`;
- соединять таблицы через `LEFT JOIN`;
- находить пользователей без заказов;
- считать стоимость заказа через выражение `price * quantity`;
- выполнять простые аналитические запросы по пользователям и заказам.

1. Простая бизнес-история

Представим небольшой интернет-магазин.

У нас уже есть таблица `users`. В ней хранятся пользователи:

id	name	email	age	country	balance
1	Ivan	ivan@example.com	25	RUS	100000.00
2	Gleb	gleb@example.com	26	RUS	110000.00
3	Sergey	sergey@example.com	28	RUS	90000.00
4	Andrew	andrew@example.com	24	RUS	95000.00
6	Tom	tom@example.com	29	USA	110000.00

Теперь нам нужна таблица заказов `orders`.

Один пользователь может сделать много заказов:

```
Пользователь Ivan
-> заказ Product A
```

-> заказ Product D

Пользователь Gleb

-> заказ Product B

-> заказ Product C

-> заказ Product E

Пользователь Sergey

-> заказов пока нет

Если хранить всё в одной таблице, данные начнут повторяться. Например, email пользователя будет дублироваться в каждой строке заказа. Поэтому мы разделяем данные:

- `users` — кто покупает;
- `orders` — что покупают;
- `orders.user_id` — ссылка на пользователя из таблицы `users`.

2. Визуальная схема связи таблиц

users			orders			
id	name		id	user_id	product_name	
1	Ivan	<-----	1	1	Product A	
2	Gleb	<-----	2	2	Product B	
3	Sergey		3	2	Product C	

Здесь:

- `users.id` — уникальный номер пользователя;
- `orders.user_id` — номер пользователя, который сделал заказ;
- связь строится по правилу: `users.id = orders.user_id`.

Простыми словами:

Если `orders.user_id = 2`, значит заказ принадлежит пользователю, у которого `users.id = 2`.

3. Подготовка к работе

Перед началом убедитесь, что вы находитесь внутри MySQL:

```
mysql>
```

Выберите базу данных:

```
USE my_database;
```

Проверьте, что таблица `users` существует:

```
SHOW TABLES;
```

Ожидаемый результат:

```
+-----+
| Tables_in_my_database |
+-----+
| users                  |
+-----+
```

Проверьте структуру таблицы `users` :

```
DESCRIBE users;
```

Важно, чтобы в таблице `users` был столбец `id` . Именно к нему будет обращаться таблица `orders` .

Часть 1. Создание таблицы `orders`

4. Если таблица `orders` уже создавалась

Если вы повторяете занятие не первый раз, таблица `orders` уже может существовать. Для чистого старта её можно удалить:

```
DROP TABLE IF EXISTS orders;
```

Эта команда удалит таблицу `orders` , если она существует.

Важно: команда удаляет таблицу вместе с её данными. В учебной базе это допустимо, но в реальной рабочей базе такие команды выполняют только осознанно и после согласования.

5. Создать таблицу заказов

```
CREATE TABLE orders (
  id INT NOT NULL AUTO_INCREMENT,
  user_id INT NOT NULL,
  product_name VARCHAR(100) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  quantity INT NOT NULL,
  order_date DATE NOT NULL,
  PRIMARY KEY (id),
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Разбор структуры:

Поле	Тип	Что хранит
<code>id</code>	<code>INT</code>	Уникальный номер заказа
<code>user_id</code>	<code>INT</code>	Номер пользователя, который сделал заказ
<code>product_name</code>	<code>VARCHAR(100)</code>	Название товара
<code>price</code>	<code>DECIMAL(10, 2)</code>	Цена одной единицы товара
<code>quantity</code>	<code>INT</code>	Количество товара в заказе
<code>order_date</code>	<code>DATE</code>	Дата заказа

Разбор ограничений:

Конструкция	Что означает
PRIMARY KEY (id)	Поле <code>id</code> уникально идентифицирует каждый заказ
FOREIGN KEY (user_id)	Поле <code>user_id</code> является внешним ключом
REFERENCES users(id)	Значение <code>user_id</code> должно существовать в <code>users.id</code>

Простыми словами: база не должна позволить создать заказ для несуществующего пользователя.

6. Почему цена задана как `DECIMAL`, а не `FLOAT`

В исходном файле поле `price` было задано как `FLOAT`:

```
price FLOAT NOT NULL
```

Для учебного примера это может работать, но для денег лучше использовать `DECIMAL(10, 2)`.

Причина простая:

- `FLOAT` — приблизительный тип числа;
- `DECIMAL` — точный тип числа;
- для цен, балансов и финансовых значений лучше использовать точный тип.

В этом файле используем:

```
price DECIMAL(10, 2) NOT NULL
```

Это означает: число максимум до 10 цифр, из них 2 цифры после запятой.

Примеры подходящих значений:

```
10.00
1250.50
99999999.99
```

7. Проверить структуру таблицы

```
DESCRIBE orders;
```

Ожидаемый результат:

```
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| id         | int           | NO   | PRI | NULL    | auto_increment |
| user_id    | int           | NO   | MUL | NULL    |                |
| product_name | varchar(100)  | NO   |     | NULL    |                |
| price      | decimal(10,2) | NO   |     | NULL    |                |
| quantity   | int           | NO   |     | NULL    |                |
| order_date | date          | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
```

Обратите внимание на поле `user_id`. В колонке `Key` может быть значение `MUL`. Это нормально: MySQL показывает, что поле участвует в индексе, связанном с внешним ключом.

8. Мини-задание 1

Ответьте устно или письменно:

1. Какая таблица является таблицей пользователей?
2. Какая таблица является таблицей заказов?
3. Какое поле связывает заказ с пользователем?
4. Почему заказу нужен `user_id`?
5. Что произойдёт, если попытаться создать заказ для пользователя, которого нет в таблице `users`?

Часть 2. Добавление заказов

9. Добавить один заказ

Добавим заказ для пользователя с `id = 1`, то есть для Ivan.

```
INSERT INTO orders (user_id, product_name, price, quantity, order_date)
VALUES (1, 'Product A', 10.00, 2, '2023-02-22');
```

Разбор:

Часть команды	Что означает
<code>INSERT INTO orders</code>	Добавляем строку в таблицу заказов
<code>user_id</code>	Указываем, кто сделал заказ
<code>product_name</code>	Название товара
<code>price</code>	Цена одной единицы товара
<code>quantity</code>	Количество
<code>order_date</code>	Дата заказа

Стоимость этого заказа можно посчитать так:

```
10.00 * 2 = 20.00
```

10. Проверить таблицу `orders`

```
SELECT *
FROM orders;
```

Ожидаемый результат:

```
+-----+-----+-----+-----+-----+-----+
| id | user_id | product_name | price | quantity | order_date |
+-----+-----+-----+-----+-----+-----+

```

	1		1		Product A		10.00		2		2023-02-22	
+	---	+	-----	+	-----	+	-----	+	---	+	-----	+

11. Добавить несколько заказов одной командой

```
INSERT INTO orders (user_id, product_name, price, quantity, order_date)
VALUES
    (2, 'Product B', 20.00, 1, '2023-02-20'),
    (2, 'Product C', 15.00, 3, '2023-02-23'),
    (1, 'Product D', 12.00, 2, '2023-02-25'),
    (2, 'Product E', 25.00, 1, '2023-02-26');
```

Здесь:

- `user_id = 1` — заказы Ivan;
- `user_id = 2` — заказы Gleb;
- у Sergey, Andrew и Tom заказов пока нет.

12. Посмотреть все заказы

```
SELECT *
FROM orders;
```

Ожидаемый результат:

+	---	+	-----	+	-----	+	-----	+	-----	+	-----	+
	id		user_id		product_name		price		quantity		order_date	
+	---	+	-----	+	-----	+	-----	+	-----	+	-----	+
	1		1		Product A		10.00		2		2023-02-22	
	2		2		Product B		20.00		1		2023-02-20	
	3		2		Product C		15.00		3		2023-02-23	
	4		1		Product D		12.00		2		2023-02-25	
	5		2		Product E		25.00		1		2023-02-26	
+	---	+	-----	+	-----	+	-----	+	-----	+	-----	+

13. Проверить работу внешнего ключа

Попробуем добавить заказ для пользователя, которого нет в таблице `users`:

```
INSERT INTO orders (user_id, product_name, price, quantity, order_date)
VALUES (999, 'Product X', 50.00, 1, '2023-02-27');
```

Ожидаемая логика: MySQL должен отказать, потому что пользователя с `id = 999` нет.

Пример ошибки может выглядеть так:

```
Cannot add or update a child row: a foreign key constraint fails
```

Это хорошая ошибка. Она защищает данные от ситуации, когда в заказах есть ссылка на несуществующего пользователя.

14. Мини-задание 2

Выполните задания:

1. Выведите все заказы.
2. Найдите заказы пользователя с `user_id = 1`.
3. Найдите заказы пользователя с `user_id = 2`.
4. Попробуйте добавить заказ с `user_id = 999` и объясните, почему возникает ошибка.

Подсказка:

```
SELECT *  
FROM orders  
WHERE user_id = 1;
```

Часть 3. Вычисляемые столбцы

15. Посчитать стоимость каждого заказа

В таблице есть цена одной единицы товара и количество. Чтобы получить итоговую стоимость строки заказа, используем выражение:

```
price * quantity
```

SQL-запрос:

```
SELECT  
    id,  
    product_name,  
    price,  
    quantity,  
    price * quantity AS order_total  
FROM orders;
```

Ожидаемый результат:

```
+-----+-----+-----+-----+-----+  
| id | product_name | price | quantity | order_total |  
+-----+-----+-----+-----+-----+  
| 1 | Product A | 10.00 | 2 | 20.00 |  
| 2 | Product B | 20.00 | 1 | 20.00 |  
| 3 | Product C | 15.00 | 3 | 45.00 |  
| 4 | Product D | 12.00 | 2 | 24.00 |  
| 5 | Product E | 25.00 | 1 | 25.00 |  
+-----+-----+-----+-----+-----+
```

Разбор:

Часть	Что означает
<code>price * quantity</code>	Умножаем цену на количество
<code>AS order_total</code>	Даём вычисленному столбцу понятное имя

`AS` создаёт псевдоним столбца. Он не меняет таблицу, а только задаёт имя в результате запроса.

16. Мини-задание 3

Напишите запрос, который выводит:

- `product_name`;
 - `quantity`;
 - `price`;
 - итоговую стоимость заказа с названием `total_price`.
-

Часть 4. Подзапросы

17. Что такое подзапрос

Подзапрос — это запрос внутри другого запроса.

Простая логика:

Сначала внутренний запрос находит значение.
Потом внешний запрос использует это значение.

Например, задача:

Найти все заказы пользователя Gleb.

Можно решить в два шага.

Шаг 1. Найти `id` пользователя Gleb:

```
SELECT id
FROM users
WHERE name = 'Gleb';
```

Результат:

```
+----+
| id |
+----+
|  2 |
+----+
```

Шаг 2. Найти заказы, где `user_id = 2`:

```
SELECT *
FROM orders
WHERE user_id = 2;
```

Подзапрос позволяет объединить эти два шага в один.

18. Найти заказы пользователя через подзапрос

```
SELECT *
FROM orders
WHERE user_id = (
    SELECT id
    FROM users
    WHERE name = 'Gleb'
);
```

Как это читается:

Покажи все заказы,
где user_id равен id пользователя,
у которого name = 'Gleb'.

Ожидаемый результат:

id	user_id	product_name	price	quantity	order_date
2	2	Product B	20.00	1	2023-02-20
3	2	Product C	15.00	3	2023-02-23
5	2	Product E	25.00	1	2023-02-26

19. Важное замечание про подзапрос с =

В запросе выше используется знак =:

```
WHERE user_id = (...)
```

Это означает, что внутренний запрос должен вернуть одно значение.

Если внутренний запрос вернёт несколько строк, MySQL выдаст ошибку.

Например, если в таблице будет несколько пользователей с именем Ivan, такой запрос может стать проблемным:

```
SELECT *
FROM orders
WHERE user_id = (
    SELECT id
    FROM users
    WHERE name = 'Ivan'
);
```

Поэтому в реальной работе лучше искать по более точному признаку, например по email:

```
SELECT *
FROM orders
WHERE user_id = (
    SELECT id
    FROM users
    WHERE email = 'gleb@example.com'
);
```

20. Подзапрос с `IN`

Если внутренний запрос может вернуть несколько значений, используем `IN`.

Задача:

Найти заказы всех пользователей из страны `RUS`.

Сначала можно найти пользователей из `RUS`:

```
SELECT id
FROM users
WHERE country = 'RUS';
```

Потом найти их заказы:

```
SELECT *
FROM orders
WHERE user_id IN (
    SELECT id
    FROM users
    WHERE country = 'RUS'
);
```

Разница:

Оператор	Когда использовать
<code>=</code>	Когда подзапрос возвращает одно значение
<code>IN</code>	Когда подзапрос может вернуть несколько значений

21. Мини-задание 4

Напишите запросы:

- Найдите все заказы пользователя `Ivan` через подзапрос.
- Найдите все заказы пользователя с email `gleb@example.com`.
- Найдите все заказы пользователей из страны `RUS`.
- Объясните разницу между `=` и `IN` в подзапросе.

Часть 5. `UNION`

22. Что делает `UNION`

`UNION` объединяет результаты двух или нескольких `SELECT`-запросов в один общий результат.

Важно: `UNION` не склеивает таблицы физически. Он только показывает общий результат запроса.

Простая аналогия:

```
SELECT 1 дал один список строк.
SELECT 2 дал другой список строк.
UNION показал их вместе.
```

23. Главное правило UNION

У объединяемых запросов должно быть одинаковое количество столбцов.

Например, этот вариант правильный:

```
SELECT id, name, 'user' AS source
FROM users
UNION
SELECT id, product_name, 'order' AS source
FROM orders;
```

В обоих запросах три столбца:

Запрос	Столбец 1	Столбец 2	Столбец 3
Из <code>users</code>	<code>id</code>	<code>name</code>	<code>'user' AS source</code>
Из <code>orders</code>	<code>id</code>	<code>product_name</code>	<code>'order' AS source</code>

Ожидаемый результат:

```
+----+-----+-----+
| id | name      | source |
+----+-----+-----+
| 1  | Ivan      | user   |
| 2  | Gleb      | user   |
| 3  | Sergey    | user   |
| 4  | Andrew    | user   |
| 6  | Tom       | user   |
| 1  | Product A | order  |
| 2  | Product B | order  |
| 3  | Product C | order  |
| 4  | Product D | order  |
| 5  | Product E | order  |
+----+-----+-----+
```

Обратите внимание: во втором столбце в результате стоит имя `name`, потому что MySQL берёт названия столбцов из первого `SELECT`.

24. Более понятный вариант с одинаковыми псевдонимами

Чтобы результат был понятнее, можно явно назвать столбцы:

```
SELECT
    id,
    name AS title,
    'user' AS source
FROM users
UNION
SELECT
    id,
    product_name AS title,
    'order' AS source
FROM orders;
```

Теперь второй столбец логически называется `title`: это может быть имя пользователя или название товара.

25. UNION и UNION ALL

Есть два похожих варианта:

Команда	Что делает
UNION	Объединяет результаты и убирает полные дубликаты строк
UNION ALL	Объединяет результаты и оставляет все строки, включая дубликаты

Для обучения можно запомнить так:

UNION – аккуратно объединить без повторов.
UNION ALL – объединить всё как есть.

В реальной аналитике `UNION ALL` часто быстрее и прозрачнее, если вы точно знаете, что дубликаты не нужно удалять.

26. Мини-задание 5

Напишите запросы:

- Объедините список имён пользователей и список названий продуктов.
- Добавьте столбец `source`, чтобы было понятно, откуда пришла строка.
- Попробуйте заменить `UNION` на `UNION ALL`.
- Объясните, почему у двух `SELECT` должно быть одинаковое количество столбцов.

Часть 6. INNER JOIN

27. Что такое JOIN

`JOIN` нужен, когда данные находятся в разных таблицах, но нам нужен общий результат.

Например, таблица `orders` хранит `user_id`, но не хранит имя пользователя. Чтобы увидеть имя пользователя рядом с заказом, нужно соединить `orders` с `users`.

28. Простая схема INNER JOIN

```
users.id = orders.user_id
```

users			orders		
id	name		id	user_id	product
1	Ivan	<-----	1	1	Product A
2	Gleb	<-----	2	2	Product B
3	Serg				

`INNER JOIN` возвращает только те строки, где найдено совпадение в обеих таблицах.

То есть пользователь без заказов в результат не попадёт.

29. Синтаксис INNER JOIN

Общий шаблон:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

В нашем случае:

```
SELECT
    users.name,
    users.email,
    orders.product_name,
    orders.order_date
FROM users
INNER JOIN orders
ON users.id = orders.user_id;
```

Ожидаемый результат:

```
+-----+-----+-----+-----+
| name | email           | product_name | order_date |
+-----+-----+-----+-----+
| Ivan | ivan@example.com | Product A    | 2023-02-22 |
| Gleb | gleb@example.com | Product B    | 2023-02-20 |
| Gleb | gleb@example.com | Product C    | 2023-02-23 |
| Ivan | ivan@example.com | Product D    | 2023-02-25 |
| Gleb | gleb@example.com | Product E    | 2023-02-26 |
+-----+-----+-----+-----+
```

В результате нет Sergey, Andrew и Tom, потому что у них нет заказов.

30. То же самое через короткие псевдонимы таблиц

В реальных SQL-запросах часто используют короткие имена таблиц:

```
SELECT
    u.name,
    u.email,
    o.product_name,
    o.order_date
FROM users AS u
INNER JOIN orders AS o
ON u.id = o.user_id;
```

Здесь:

Псевдоним	Что означает
u	таблица <code>users</code>
o	таблица <code>orders</code>

Так запрос становится короче и удобнее для чтения.

31. Добавить стоимость заказа в JOIN

```
SELECT
    u.name,
    u.email,
    o.product_name,
    o.price,
    o.quantity,
    o.price * o.quantity AS order_total,
    o.order_date
FROM users AS u
INNER JOIN orders AS o
ON u.id = o.user_id;
```

Ожидаемый результат:

name	email	product_name	price	quantity	order_total	order_date
Ivan	ivan@example.com	Product A	10.00	2	20.00	2023-02-22
Gleb	gleb@example.com	Product B	20.00	1	20.00	2023-02-20
Gleb	gleb@example.com	Product C	15.00	3	45.00	2023-02-23
Ivan	ivan@example.com	Product D	12.00	2	24.00	2023-02-25
Gleb	gleb@example.com	Product E	25.00	1	25.00	2023-02-26

Такой результат уже похож на аналитическую выгрузку: видно пользователя, товар, дату и сумму строки заказа.

32. Мини-задание 6

Напишите запросы:

1. Выведите имя пользователя и название товара через `INNER JOIN`.
2. Добавьте дату заказа.
3. Добавьте итоговую стоимость заказа.
4. Оставьте только заказы пользователя `Gleb`.

Подсказка для последнего пункта:

```
WHERE u.name = 'Gleb'
```

Полный запрос можно строить так:

```
SELECT ...
FROM users AS u
INNER JOIN orders AS o
ON u.id = o.user_id
WHERE ...;
```

33. Что делает `LEFT JOIN`

`LEFT JOIN` возвращает:

- все строки из левой таблицы;
- совпавшие строки из правой таблицы;
- `NULL`, если совпадения справа нет.

Главное правило:

`LEFT JOIN` сохраняет все строки из таблицы, которая написана слева от `LEFT JOIN`.

34. Сравнение `INNER JOIN` и `LEFT JOIN`

Тип соединения	Что возвращает	Простыми словами
<code>INNER JOIN</code>	Только совпавшие строки	Только пользователи, у которых есть заказы
<code>LEFT JOIN</code>	Все строки из левой таблицы и совпадения справа	Все пользователи, даже если заказов нет

35. Запрос с `LEFT JOIN`

```
SELECT
    users.name,
    users.email,
    orders.product_name,
    orders.order_date
FROM users
LEFT JOIN orders
ON users.id = orders.user_id;
```

Ожидаемый результат:

```
+-----+-----+-----+-----+
| name | email           | product_name | order_date |
+-----+-----+-----+-----+
| Ivan | ivan@example.com | Product A    | 2023-02-22 |
| Ivan | ivan@example.com | Product D    | 2023-02-25 |
| Gleb | gleb@example.com | Product B    | 2023-02-20 |
| Gleb | gleb@example.com | Product C    | 2023-02-23 |
| Gleb | gleb@example.com | Product E    | 2023-02-26 |
| Sergey | sergey@example.com | NULL        | NULL        |
| Andrew | andrew@example.com | NULL        | NULL        |
| Tom   | tom@example.com  | NULL        | NULL        |
+-----+-----+-----+-----+
```

`NULL` означает отсутствие значения.

В нашем случае:

`NULL` в `product_name` и `order_date` означает, что у пользователя нет заказа.

36. Найти пользователей без заказов

Это частая аналитическая задача.

Например:

Найти клиентов, которые зарегистрировались, но ничего не купили.

Запрос:

```
SELECT
    u.id,
    u.name,
    u.email
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id
WHERE o.id IS NULL;
```

Как это читается:

Покажи всех пользователей слева,
приклей к ним заказы, если они есть,
а потом оставь только тех,
у кого заказа не нашлось.

Ожидаемый результат:

```
+----+-----+-----+
| id | name  | email                |
+----+-----+-----+
|  3 | Sergey | sergey@example.com |
|  4 | Andrew | andrew@example.com  |
|  6 | Tom    | tom@example.com     |
+----+-----+-----+
```

37. Важное правило про условия в LEFT JOIN

Слушатели часто допускают ошибку: пишут `LEFT JOIN`, а потом условием в `WHERE` случайно превращают его по смыслу в `INNER JOIN`.

Например:

```
SELECT
    u.name,
    o.product_name,
    o.order_date
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id
WHERE o.order_date >= '2023-02-23';
```

Такой запрос уберёт пользователей без заказов, потому что у них `o.order_date` равен `NULL`.

Для начального уровня достаточно запомнить:

Если хотим сохранить пользователей без заказов, аккуратно работаем с условиями по правой таблице.

38. Мини-задание 7

Напишите запросы:

1. Выведите всех пользователей и их заказы через `LEFT JOIN`.
2. Найдите пользователей без заказов.
3. Объясните, почему у некоторых строк в результате появляются `NULL`.
4. Сравните результат `INNER JOIN` и `LEFT JOIN`.

Часть 8. Простая аналитика по заказам

39. Посчитать количество заказов по пользователям

```
SELECT
    u.name,
    COUNT(o.id) AS orders_count
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id
GROUP BY u.id, u.name;
```

Ожидаемый результат:

name	orders_count
Ivan	2
Gleb	3
Sergey	0
Andrew	0
Tom	0

Почему используется `COUNT(o.id)`, а не `COUNT(*)`:

- `COUNT(*)` считает строки результата после `LEFT JOIN`;
- у пользователя без заказов всё равно будет одна строка с `NULL`;
- `COUNT(o.id)` считает только найденные заказы, потому что `NULL` не считается.

40. Посчитать сумму заказов по пользователям

```
SELECT
    u.name,
    COALESCE(SUM(o.price * o.quantity), 0) AS total_spent
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id
GROUP BY u.id, u.name;
```

Ожидаемый результат:

```
+-----+-----+
| name  | total_spent |
+-----+-----+
| Ivan  |      44.00 |
| Gleb  |      90.00 |
| Sergey|       0.00 |
| Andrew|       0.00 |
| Tom   |       0.00 |
+-----+-----+
```

Здесь появляется новая функция `COALESCE`.

Простое объяснение:

`COALESCE(значение, 0)` означает:
если значение есть — покажи его,
если там `NULL` — покажи `0`.

Если у пользователя нет заказов, сумма будет `NULL`. Для отчёта удобнее показать `0`.

41. Отсортировать пользователей по сумме заказов

```
SELECT
    u.name,
    COALESCE(SUM(o.price * o.quantity), 0) AS total_spent
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id
GROUP BY u.id, u.name
ORDER BY total_spent DESC;
```

Так мы получаем простой рейтинг пользователей по сумме заказов.

42. Мини-задание 8

Напишите запросы:

1. Посчитайте количество заказов каждого пользователя.
2. Посчитайте сумму заказов каждого пользователя.
3. Отсортируйте пользователей по сумме заказов от большего к меньшему.
4. Найдите пользователя с максимальной суммой заказов.

Часть 9. Типичные ошибки новичков

Ошибка	Почему возникает	Как исправить
Cannot add or update a child row	В <code>orders.user_id</code> указали пользователя, которого нет в <code>users.id</code>	Проверить пользователя через <code>SELECT * FROM users;</code>
Unknown column	Ошибка в названии столбца	Проверить структуру через <code>DESCRIBE table_name;</code>

Ошибка	Почему возникает	Как исправить
Table 'orders' already exists	Таблица уже была создана раньше	Использовать <code>DROP TABLE IF EXISTS orders;</code> для учебного повтора
Подзапрос с <code>=</code> вернул много строк	<code>=</code> ждёт одно значение	Использовать более точное условие или оператор <code>IN</code>
<code>JOIN</code> вернул слишком много строк	У пользователя несколько заказов, поэтому строки размножились	Проверить связь один-ко-многим: один пользователь — много заказов
Пользователи без заказов пропали	Использовали <code>INNER JOIN</code> вместо <code>LEFT JOIN</code>	Использовать <code>LEFT JOIN</code>
Не понимают <code>NULL</code>	<code>NULL</code> воспринимают как пустую строку или ноль	Объяснить: <code>NULL</code> означает отсутствие значения
Сумма по пользователю без заказов равна <code>NULL</code>	<code>SUM</code> по отсутствующим значениям возвращает <code>NULL</code>	Использовать <code>COALESCE(SUM(...), 0)</code>

Часть 10. Итоговая практика

43. Задания для самостоятельной работы

Выполните задания последовательно.

Задание 1

Проверьте, что таблицы `users` и `orders` существуют.

Задание 2

Выведите все данные из таблицы `orders`.

Задание 3

Выведите все заказы пользователя с `user_id = 2`.

Задание 4

Выведите название товара, цену, количество и итоговую стоимость заказа.

Задание 5

Найдите все заказы пользователя `Gleb` через подзапрос.

Задание 6

Найдите все заказы пользователей из страны `RUS` через подзапрос с `IN`.

Задание 7

Объедините список пользователей и список продуктов через `UNION`.

Задание 8

Выведите имя пользователя, email, название товара и дату заказа через `INNER JOIN`.

Задание 9

Выведите всех пользователей и их заказы через `LEFT JOIN`.

Задание 10

Найдите пользователей без заказов.

Задание 11

Посчитайте количество заказов по каждому пользователю.

Задание 12

Посчитайте сумму заказов по каждому пользователю и отсортируйте результат по убыванию суммы.

Часть 11. Ответы-подсказки к итоговой практике

Задание 1

```
SHOW TABLES;
```

Задание 2

```
SELECT *  
FROM orders;
```

Задание 3

```
SELECT *  
FROM orders  
WHERE user_id = 2;
```

Задание 4

```
SELECT  
    product_name,  
    price,  
    quantity,  
    price * quantity AS order_total  
FROM orders;
```

Задание 5

```
SELECT *  
FROM orders  
WHERE user_id = (  
    SELECT id  
    FROM users  
    WHERE name = 'Gleb'  
);
```

Задание 6

```
SELECT *
FROM orders
WHERE user_id IN (
    SELECT id
    FROM users
    WHERE country = 'RUS'
);
```

Задание 7

```
SELECT
    id,
    name AS title,
    'user' AS source
FROM users
UNION
SELECT
    id,
    product_name AS title,
    'order' AS source
FROM orders;
```

Задание 8

```
SELECT
    u.name,
    u.email,
    o.product_name,
    o.order_date
FROM users AS u
INNER JOIN orders AS o
ON u.id = o.user_id;
```

Задание 9

```
SELECT
    u.name,
    u.email,
    o.product_name,
    o.order_date
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id;
```

Задание 10

```
SELECT
    u.id,
    u.name,
    u.email
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id
WHERE o.id IS NULL;
```

Задание 11

```
SELECT
    u.name,
    COUNT(o.id) AS orders_count
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id
GROUP BY u.id, u.name;
```

Задание 12

```
SELECT
    u.name,
    COALESCE(SUM(o.price * o.quantity), 0) AS total_spent
FROM users AS u
LEFT JOIN orders AS o
ON u.id = o.user_id
GROUP BY u.id, u.name
ORDER BY total_spent DESC;
```

Часть 12. Шпаргалка по командам файла

Задача	Команда или конструкция
Создать таблицу заказов	CREATE TABLE orders (...)
Задать внешний ключ	FOREIGN KEY (user_id) REFERENCES users(id)
Добавить заказ	INSERT INTO orders (...) VALUES (...)
Посчитать стоимость строки заказа	price * quantity AS order_total
Найти данные через подзапрос	WHERE user_id = (SELECT id FROM users ...)
Проверить список значений	WHERE user_id IN (...)
Объединить результаты запросов	UNION
Объединить результаты с дублями	UNION ALL
Соединить только совпавшие строки	INNER JOIN
Сохранить все строки слева	LEFT JOIN
Найти строки без совпадений справа	WHERE right_table.id IS NULL
Посчитать количество	COUNT(...)
Посчитать сумму	SUM(...)

Задача	Команда или конструкция
Заменить <code>NULL</code> на 0	<code>COALESCE(value, 0)</code>
Сгруппировать результат	<code>GROUP BY</code>
Отсортировать результат	<code>ORDER BY ... DESC</code>

Часть 13. Контрольные вопросы для преподавателя

Эти вопросы можно задать группе устно.

1. Зачем разделять пользователей и заказы на разные таблицы?
2. Что означает связь `users.id = orders.user_id`?
3. Что такое внешний ключ?
4. Почему нельзя добавить заказ для несуществующего пользователя?
5. Что делает подзапрос?
6. Чем `=` отличается от `IN` в подзапросах?
7. Что делает `UNION`?
8. Почему у запросов в `UNION` должно быть одинаковое количество столбцов?
9. Что возвращает `INNER JOIN`?
10. Что возвращает `LEFT JOIN`?
11. Что означает `NULL` в результате `LEFT JOIN`?
12. Как найти пользователей без заказов?
13. Почему для подсчёта заказов лучше использовать `COUNT(o.id)`, а не `COUNT(*)`?
14. Зачем нужна функция `COALESCE`?

Часть 14. Итог занятия по файлу

В этом файле мы перешли от одной таблицы к связанной структуре данных:

```
users
-> связаны через users.id = orders.user_id
orders
```

Мы научились:

- создавать таблицу с внешним ключом;
- добавлять заказы;
- понимать ошибку внешнего ключа;
- считать стоимость заказа;
- использовать подзапросы;
- объединять результаты через `UNION`;
- соединять таблицы через `INNER JOIN` и `LEFT JOIN`;
- находить пользователей без заказов;
- считать количество и сумму заказов по пользователям.

Это базовый уровень SQL, который нужен аналитику данных для работы с реальными корпоративными базами: CRM, интернет-магазинами, банковскими системами, системами учёта и логами операций.

После этого материала можно переходить к связке SQL и Python: получать результат SQL-запроса и анализировать его в pandas.